# Portofolio

**Introduction**

This portfolio demonstrates an end-to-end approach to software testing, covering test strategy design, test case development, and automation implementation using Selenium- and Appium-based frameworks. The automation framework adopts a hybrid approach (data-driven and keyword-driven) and is designed to support dynamic and scalable test execution.

**Key Highlights**

- Hybrid Automation Framework (Keyword-Driven & Data-Driven)
- Dynamic Test Execution using ExecuteTable
- Custom Conditional Keywords for handling non-linear test flows
- Modular and Scalable Framework Architecture
- Clear separation of Script, Test Data, and Locators
- Dual Reporting System (Summary and Detailed Reports)

**Project Overview**

**Project Title:** Web Appointment Booking Testing

This project focuses on testing a web-based appointment booking application that includes core features such as login, appointment creation, history management, and logout. The objective of this project is to ensure the reliability of core functionalities and improve testing efficiency through automation. In this project, I was responsible for designing test strategies, developing test cases, and implementing an automation framework to support scalable and maintainable test execution.

**Problem - Solution - Impact**

Lack of Flexibility & Reusability

Problem

Automation tests were not flexible and required code changes for different scenarios, while test scripts were repetitive and difficult to maintain.

Solution

Designed and implemented a modular automation framework using a hybrid approach (data-driven and keyword-driven), with clear separation of scripts, test data, and locators.

Impact

Improved test reusability and flexibility, reduced maintenance effort, and enabled scalable automation development.

Automation Instability (Timing & Locator Issues)

Problem

Automation tests were unstable due to timing issues and frequently changing element locators, resulting in flaky test results and unreliable execution.

Solution

Enhanced synchronization using explicit waits and improved locator strategies by implementing more stable and maintainable selectors with centralized management.

Impact

Improved test stability and reliability, reduced flaky failures, and minimized maintenance effort.

**Scope of Testing** (Functionality application)

- Login
- Appointment booking
- View history
- Logout

**Tools & Technology**

- Automation Tool: Selenium
- Programming Language: Java
- Test Framework: TestNG
- Data Driven: Excel
- Version Control: Git

**Test Strategy**

The testing approach combines both manual and automation testing, with execution determined by the value and characteristics of each test scenario.

The testing scope includes:

- Core functionalities (Login & Appointment Booking)
- Validation testing for input handling
- Edge case testing for extreme and unexpected conditions

Automation implementation is determined based on the following considerations:

- Frequency (usage frequency of the feature)
- Criticality (business impact)
- Stability (rate of system changes)
- Effort (automation complexity and maintenance cost)

**Test Scenarios**

Login Module

- TS-01: User logs in with valid credentials
- TS-02: User fails to log in with an incorrect password
- TS-03: User fails to log in without entering a username
- TS-04: User fails to log in without entering a password
- TS-05: User fails to log in with all fields empty
- TS-06: User attempts to log in with invalid characters (security / edge case)

Appointment Module

- TS-07: User successfully creates an appointment with valid data
- TS-08: User fails to create an appointment without selecting a date
- TS-09: User fails to create an appointment without selecting a facility
- TS-10: User can create an appointment without a comment (optional field)
- TS-11: User fails to create an appointment with a past date

History Module

- TS-12: User can view booking history after creating an appointment
- TS-13: User sees an empty state when no history is available

Logout Module

- TS-14: User successfully logs out

## Test case

| TC ID | Module | Scenario | Type | Priority | Precondition | Steps | Test Data |
|---|---|---|---|---|---|---|---|
| TC-01 | Login | Login with valid credentials | Positive | High | User is on login page | 1. Enter valid username<br>2. Enter valid password<br>3. Click login | Valid user |
| TC-02 | Login | Login with incorrect password | Negative | High | User is on login page | 1. Enter valid username<br>2. Enter incorrect password<br>3. Click login | Invalid password |
| TC-03 | Login | Login without username | Negative | High | User is on login page | 1. Leave username empty<br>2. Click login | Empty |
| TC-04 | Login | Login without password | Negative | High | User is on login page | 1. Leave password empty<br>2. Click login | Empty |
| TC-05 | Login | Login with all fields empty | Negative | High | User is on login page | 1. Leave all fields empty<br>2. Click login | Empty |
| TC-06 | Login | Login with invalid characters | Edge | Medium | User is on login page | 1. Enter invalid characters<br>2. Click login | ' OR 1=1 |
| TC-07 | Appointment | Booking with valid data | Positive | High | User is logged in | 1. Fill all required fields<br>2. Click submit | Valid data |
| TC-08 | Appointment | Booking without date | Negative | High | User is logged in | 1. Leave date empty<br>2. Click submit | Empty |
| TC-09 | Appointment | Booking without selecting facility | Negative | Medium | User is logged in | 1. Do not select facility<br>2. Click submit | Empty |
| TC-10 | Appointment | Booking without comment (optional) | Positive | Low | User is logged in | 1. Leave comment empty<br>2. Click submit | Empty |
| TC-11 | Appointment | Booking with past date | Edge | Medium | User is logged in | 1. Enter past date<br>2. Click submit | Past date |
| TC-12 | History | View history after booking | Positive | Medium | User has existing booking | 1. Navigate to history page | Valid user |
| TC-13 | History | View empty history | Negative | Low | User has no booking | 1. Navigate to history page | No data |
| TC-14 | Logout | Successful logout | Positive | Medium | User is logged in | 1. Click logout | Empty |

## Automation Priority Matrix

| Scenario | Frequency | Criticality | Stability | Effort | Priority | Reason |
|---|---|---|---|---|---|---|
| Login with valid credentials | High | High | High | Low | High | Core flow and frequently used |
| Login with incorrect password | Medium | Medium | High | Low | Medium | Important validation but not always part of regression |
| Login without username | Low | Low | High | Low | Low | Simple validation |
| Login without password | Low | Low | High | Low | Low | Simple validation |
| Login with all fields empty | Low | Low | High | Low | Low | Considered redundant |
| Login with invalid characters | Low | Medium | Medium | Medium | Low | More relevant for security testing |
| Booking with valid data | High | High | High | Medium | High | Core business flow |
| Booking without date | Medium | High | High | Low | High | Critical validation |
| Booking without selecting facility | Low | Medium | High | Low | Medium | Not business-critical |
| Booking without comment | Low | Low | High | Low | Low | Optional field |
| Login with valid credentials | High | High | High | Low | High | Core flow and frequently used |
| Login with incorrect password | Medium | Medium | High | Low | Medium | Important validation but not always part of regression |
| Login without username | Low | Low | High | Low | Low | Simple validation |

Automation Priority Categories

- **High** (Must Automate)
  Test scenarios that are critical to core business flows, frequently executed, and require consistent validation through automation.
- **Medium** (Optional / Future Automation)
  Test scenarios with moderate impact that can be automated if time and resources permit.
- **Low** (Manual Testing Preferred)
  Test scenarios with low impact or minimal risk, where manual testing is considered sufficient.

**Test Design**

Tests are designed in a modular structure to support automation and minimize script duplication, ensuring better maintainability and scalability.

Test Structure

Login Module

- LoginPositiveTest
- LoginNegativeTest

Appointment Module

- AppointmentPositiveTest
- AppointmentValidationTest
- AppointmentEdgeTest

History Module

- HistoryTest

Logout Module

- LogoutTest

Test Flow Strategy

Testing is executed using two main approaches:

1. **Independent Tests**

   Each test is designed to run independently without relying on other test cases.

   This ensures better stability and reduces dependency-related failures.

   *Example*: Appointment tests include login as part of the setup process.

2. **End-to-End (E2E) Tests**

   Used to validate the integration between multiple features and ensure the overall user flow works as expected.

   *Example*: Login → Make Appointment → View History → Logout

Test Categorization

To improve readability and maintainability, tests are grouped into the following categories:

**Positive Tests**

Use valid data to ensure core functionalities work as expected.

*Example*: Login with valid credentials, booking with valid data

**Validation Tests**

Use invalid inputs to verify that the system handles validation correctly.

*Example*: Booking without a date, login with incorrect password

**Edge Case Tests**

Cover extreme or uncommon scenarios that may expose hidden defects.

*Example*: Booking with a past date

This design ensures flexible test execution, reduces maintenance overhead, and supports scalable automation development.

**Automation Framework Design**

**1. Struktur folder**

```
main/
    application/
        APPOINMENT/
            SpecificConfig
            SpecificFunction
    framework/
        baseFunction/
            baseController
            baseFunction
            assertFunction
        factory/
        listener/
        main/
            DynamicTest
        service/
        utilities/
resource/
    makeAppointment/
        script/
            LoginScript
                LoginPositiveTest
                LoginNegativeTest
            AppointmentScript
                AppointmentPositiveTest
                AppointmentValidationTest
                AppointmentEdgeTest
            HistoryScript
                HistoryTest
            LogoutScript
                LogoutTest
        data-driven/
            LoginData
            AppointmentData
        ExecuteTable
        Locator
```

## 2. Framework Structure

The automation framework is designed using a modular approach, with a clear separation between code (logic) and resources (test data and scripts).

The overall structure is divided into two main components:

1. Main (Automation Code)

The main/ directory contains the core automation code, organized into two primary layers:

**Framework Layer**

Contains reusable and generic components used across the entire automation framework.

baseFunction/
- baseController -> Controls the overall automation execution flow
- baseFunction -> Collection of core actions (e.g., click, input, etc.)
- assertFunction -> Handles validation and assertions

factory/
- Manages object initialization (e.g., WebDriver, configuration, etc.)

listener/
- Handles test execution events such as reporting and logging

main/
- DynamicTest -> Entry point for executing tests dynamically

service/
- Contains additional logic or integration layers

utilities/
- Provides helper functions to support the framework

**Application Layer**

Contains application-specific logic and supports conditional execution through custom keywords.

*Example*:

application/
- APPOINTMENT/
  - SpecificConfig (Configuration variables for the module)
  - SpecificFunction (Custom keywords / functions)

2. Resource (Test Data & Script)

The resource/ directory stores all test data and automation scripts, structured by application/module.

*Example*:

resource/
- makeAppointment/
  - script/
  - data-driven/
  - ExecuteTable
  - Locator

**Script**
Keyword-driven test scripts used to execute automation flows
**Data-Driven**
Test data separated from scripts to support data-driven testing
**ExecuteTable**
Controls which tests are executed and defines the automation flow
**Locator**
Stores element locators (e.g., XPath) used during execution

**Reusability Strategy**
The framework is designed with a strong focus on reusability to eliminate code duplication and improve efficiency in both test development and maintenance. This approach ensures that each component can be reused across multiple scenarios without requiring redundant implementation.

Reusable Component Design

1. Base Functions (Atomic Functions)
   Base functions represent generic, low-level keywords that can be reused across all modules.
   Examples: click, set_text, get_text, select, wait_element
   Objectives:
   - Eliminate duplication of common actions
   - Enable reuse across different applications without redefining functions
2. specificFunction (Custom Function)
   Specific functions are custom keywords designed to handle application-specific logic, particularly scenarios that cannot be covered by base functions.
   Unlike atomic base functions, specific functions are capable of:
   - Introducing custom behavior
   - Handling conditional logic
   - Controlling dynamic test execution flow

Script Reusability
The framework applies a script reuse approach by defining a single main flow per feature, which can be reused across multiple scenarios by modifying test data and assertions.
*Example*:
Valid booking vs. booking without a date share the same execution flow, with differences only in input data and expected results.

**Data Strategy**

The framework adopts a data-driven testing approach, where all test data is separated from the test scripts. This approach allows new scenarios to be added without modifying the automation scripts, improving flexibility and scalability. Test data is linked to the script through value references, while the executed dataset is determined based on the scenario defined in the ExecuteTable.

Data Structure

```
resource/
        makeAppointment/
                data-driven/
                        LoginData.xlsx
                        AppointmentData.xlsx
```

Objectives:

- Improve testing flexibility
- Enable reuse of scripts with multiple data variations
- Simplify maintenance and the addition of new test scenarios

Benefits:

- A single script can cover multiple scenarios
- New test cases can be added through data only (no script changes required)
- Reduced test duplication

**Execution Strategy**

The execution strategy defines how automation tests are executed in a flexible, efficient, and scalable manner without requiring code changes. The framework adopts a configuration-driven dynamic execution approach, enabling tests to be executed at different levels:

- Per module
- Per scenario
- Full regression
- Parallel execution (optional)

Execution Modes

The framework supports multiple execution modes to accommodate different testing needs:

1. Module-Based Execution

   Tests are executed based on specific modules, such as:

   - Login only
   - Appointment only

   This mode is suitable for test development, debugging, and validating specific features in isolation.

2. Scenario-Based Execution
   Tests are executed based on scenario categories, such as:
   - Positive
   - Validation
   - Edge Cases

   This mode is useful for focused testing and targeted bug verification within specific scenarios.
3. Full Regression Execution
   Executes all available tests within the framework. This mode is typically used for regression testing before release and for integration validation.
4. Custom Execution (Dynamic Execution)
   Uses the ExecuteTable to dynamically control:
   - Which tests are executed
   - Which data sets are used
   - Which execution flow is applied

   This is the core capability of the framework, allowing fully customizable test execution based on testing needs.

ExecuteTable (Centralized Execution Control)
Execution is centrally managed through the ExecuteTable, which acts as a configuration layer for controlling test execution flow, data selection, and scenario mapping.
The ExecuteTable includes key information such as:

- Execution order
- Automation priority
- Test type
- Mapping to test cases and data-driven inputs

The ExecuteTable is structured into two main components:

- DataTable, Defines which data sets are executed
- Flow, Defines the execution flow and sequence of actions

**Datatable Configuration**
The ExecuteTable defines the execution strategy through a set of configurable columns.

| | A | B | C | D | E | F | G | H | I |
|---|---|---|---|---|---|---|---|---|---|
| 1 | No | Run | Test ID | Flow | Priority Automate | Test Case Coverage | Stagging | Status | Description |
| 2 | 1 | Yes | Test-01 | Booking Success | High | TC-01,TC-07,TC-12, TC-14 | Smoke, Regression | | |
| 3 | 2 | Yes | Test-02 | Login dengan password salah | Medium | TC-02 | Validation | | |
| 4 | 3 | Yes | Test-03 | Booking tanpa tanggal | High | TC-08 | Validation | | |
| 5 | 4 | Yes | Test-04 | Booking dengan tanggal masa lalu | High | TC-11 | Validation | | |
| 6 | | | | | | | | | |

Below is the explanation of each column:

- **No**
  Defines the execution sequence of test scenarios. This value determines the order of execution and is also linked to the corresponding flow and data set.

- **Run**
  Controls whether a test is executed or skipped:
  **Yes** Test will be executed
  **No / Empty** Test will be skipped
  The decision can be driven by automation priority or testing stage (e.g., Smoke, Regression).
- **Flow**
  Represents the name of the test flow being executed.
  This column does not directly affect execution, but is used for labeling and easier scenario identification.
- **Priority Automate**
  Defines the automation priority level:
  - High
  - Medium
  - Low

  This helps determine which scenarios should be prioritized for automation execution.
- **Test Case Coverage**
  Indicates the mapping between execution flow and the test cases it covers.
  This ensures:
  - Clear visibility of test coverage
  - No test cases are missed during execution
- **Staging**
  Defines the testing type or phase, such as:
  - Smoke
  - Regression
  - Validation
  - UAT

  This column is typically updated (e.g., highlighted) after successful execution.
- **Status**
  Represents the result of the test execution:
  - Passed
  - Failed
- **Description**
  Provides additional details about the test result, especially for failed scenarios.
  *Example*:
  "Failed – actual result does not match expected result."

**Flow Configuration**

| | A | B | C | D |
|---|---|---|---|---|
| 1 | No | Script | Feature | Data Driven |
| 2 | 1 | LoginScript | Login Success | Login |
| 3 | 1 | AppointmentScript | Appointment Success | Appointment |
| 4 | 1 | HistoryScript | View History | - |
| 5 | 1 | LogoutScript | Logout Success | |
| 6 | 2 | LoginScript | Login Failed | Login |
| 7 | 3 | LoginScript | Login Setup | Login |
| 8 | 3 | AppointmentScript | Appointment Failed | Appointment |
| 9 | 4 | LoginScript | Login Setup | Login |
| 10 | 4 | AppointmentScript | Appointment Failed | Appointment |
| 11 | | | | |

The Flow table is used to define which scripts, features, and data sets are executed as part of the automation run. Each flow represents a sequence of execution.

For example, execution may start with:
***LoginScript*** > feature ***Login Success*** > data ***Login*** followed by Appointment > History > Logout, forming a complete end-to-end flow.

This configuration allows flexible orchestration of test execution without modifying the underlying automation code.

Benefits

- Execute a subset of tests without changing the code
- Easily support regression and smoke testing
- Add new test scenarios by updating data only (no script changes required)
- Reduce test case duplication
- Accelerate regression testing execution
- Enable non-technical QA to contribute by modifying configuration (e.g., Excel)

**Bug Reporting**

Bug reports are structured to ensure they are easily reproducible and actionable for the development team. Each report provides clear, concise, and complete information to accelerate issue identification and resolution.

Bug Report Template

**Bug Title**: A clear and concise summary of the issue
**Bug ID**: A unique identifier using the format: BUG-<TC ID>-<No Bug>
**Severity**: Defines the impact of the defect on the system: Critical, High, Medium, Low
**Priority**: Defines how quickly the issue should be resolved: High, Medium, Low
**Environment**: the testing environment (e.g., browser, driver version, staging)
**Steps and Data to Reproduce**: Detailed steps to reproduce the issue (Step - Data)
**Actual Result**: The current system behavior
**Expected Result**: The expected system behavior
**Attachment**: Supporting evidence such as screenshots, screen recordings, or logs

Severity

Represents the impact of a defect on the system, *how severe the issue is from a technical perspective.*
It is determined by:

- The impact on application functionality
- Whether the feature is partially or completely broken
- Availability of a workaround

Level

- Critical, System crash, data corruption, or application is completely unusable
- High, Core functionality is not working
- Medium, Functionality works but contains defects
- Low, Minor issues such as cosmetic or UI defects (e.g., typos)

Priority

Represents the urgency of fixing a defect, *how quickly the issue should be resolved from a business perspective.*
Ditentukan oleh:

- Business urgency
- Release deadlines
- Impact on user or client
- Stakeholder requirments

Level

- High, must be fix immediately
- Medium, can be scheduled for later fixes
- Low, not urgent can be deferred

**Bug Report**

**Bug Title**: Booking allows submission with a past visit date

**Bug ID**: BUG-TC-011-001

**Severity**: Medium

**Priority**: Medium

**Environment**: Chrome, Chromedriver v145, Development Environment

**Steps and Data to Reproduce**:

***Login***

Navigate to: https://katalon-demo-cura.herokuapp.com/profile.php#login

Username: John Doe

Password: ThisIsNotAPassword

***Booking***

Facility: Hongkong CURA Healthcare Center

Healthcare Program: Medicaid

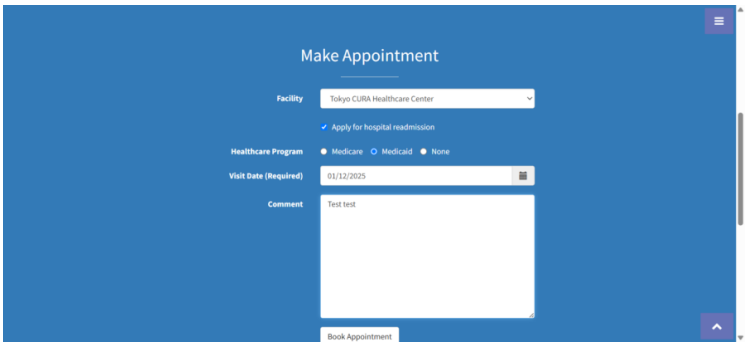
Visit Date: 01/12/2025 (past date)

Comment: Test test

**Actual Result**: The system successfully creates the appointment using a past date.

**Expected Result**: The system should reject the booking and display a validation error for past dates.

**Attachment**:



**Reporting**

The framework provides a two-layer reporting system to support both stakeholder visibility (Summary Report) and technical debugging needs (Detailed Report).

1. Excel Report (Summary Report)

   The Excel report is design to provide a high-level overview of test execute results, making it easier for stakeholders to understand the overall quality status.

   It includes:

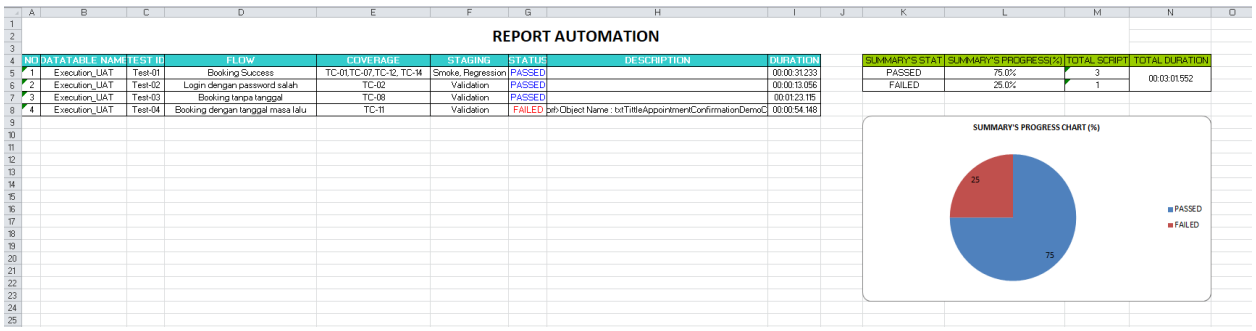
   - Test Execution Summary Table

     Overview of total test cases, passed, failed, and execution status
   - Visual Summary (Charts)

     Graphical representation of test results for quick insights
   - Failed Test Table

     List of failed test cases for quick identification of issues
   - Working Paper (WP)

     Step-by-step capture of test execution, including screenshots for each action



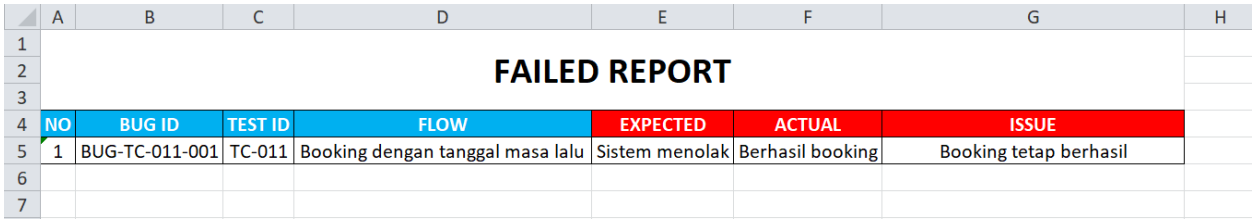
REPORT AUTOMATION

| NO | DATATABLE NAME | TEST ID | FLOW | COVERAGE | STAGING | STATUS | DESCRIPTION | DURATION |
|---|---|---|---|---|---|---|---|---|
| 1 | Execution_UAT | Test-01 | Booking Success | TC-01,TC-07,TC-12, TC-14 | Smoke, Regression | PASSED | | 00:00:31.233 |
| 2 | Execution_UAT | Test-02 | Login dengan password salah | TC-02 | Validation | PASSED | | 00:00:13.056 |
| 3 | Execution_UAT | Test-03 | Booking tanpa tanggal | TC-08 | Validation | PASSED | | 00:01:23.115 |
| 4 | Execution_UAT | Test-04 | Booking dengan tanggal masa lalu | TC-11 | Validation | FAILED | prh.Object Name : txtTittleAppointmentConfirmationDemoC | 00:00:54.148 |

| SUMMARY'S STAT | SUMMARY'S PROGRESS(%) | TOTAL SCRIPT | TOTAL DURATION |
|---|---|---|---|
| PASSED | 75.0% | 3 | 00:03:01.552 |
| FAILED | 25.0% | 1 | |



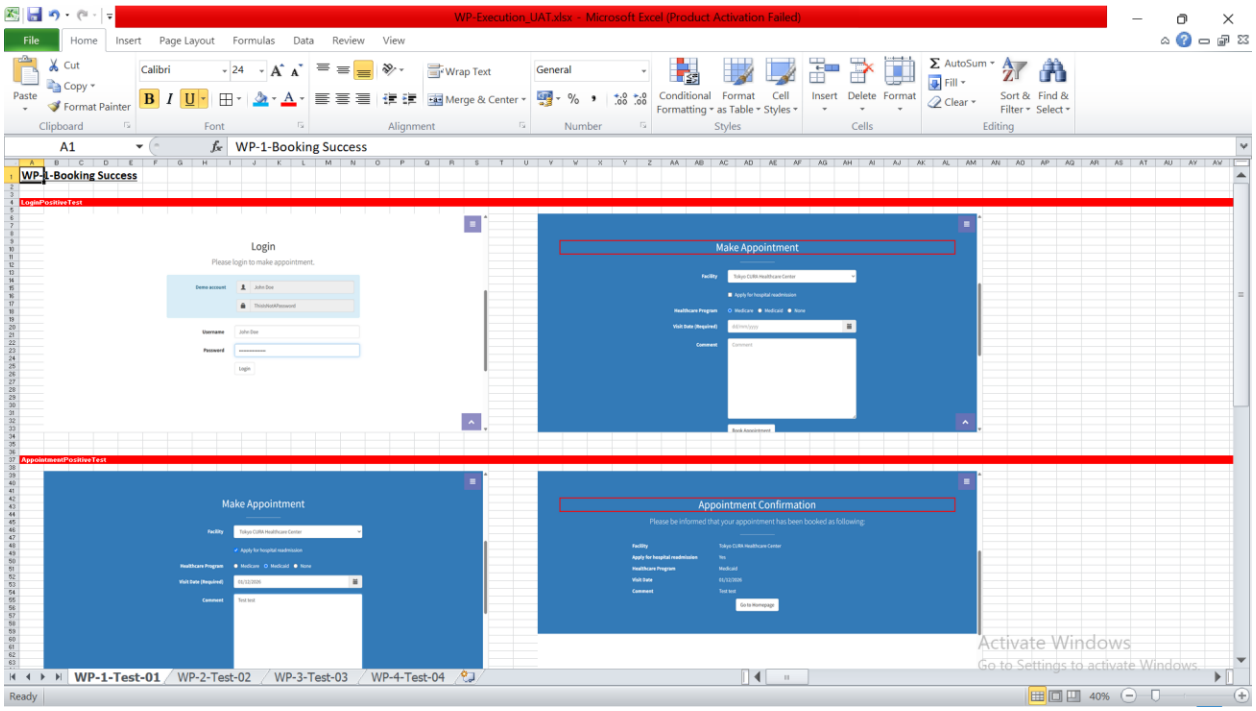
Summary Report



FAILED REPORT

| NO | BUG ID | TEST ID | FLOW | EXPECTED | ACTUAL | ISSUE |
|---|---|---|---|---|---|---|
| 1 | BUG-TC-011-001 | TC-011 | Booking dengan tanggal masa lalu | Sistem menolak | Berhasil booking | Booking tetap berhasil |

Failed Report



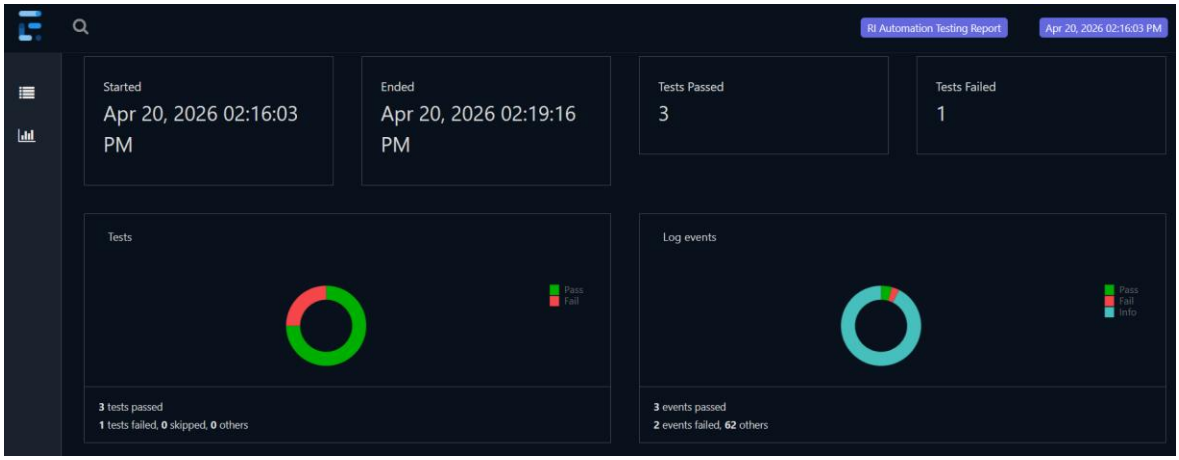

Working Paper

2. Extent Report (Detailed Report)

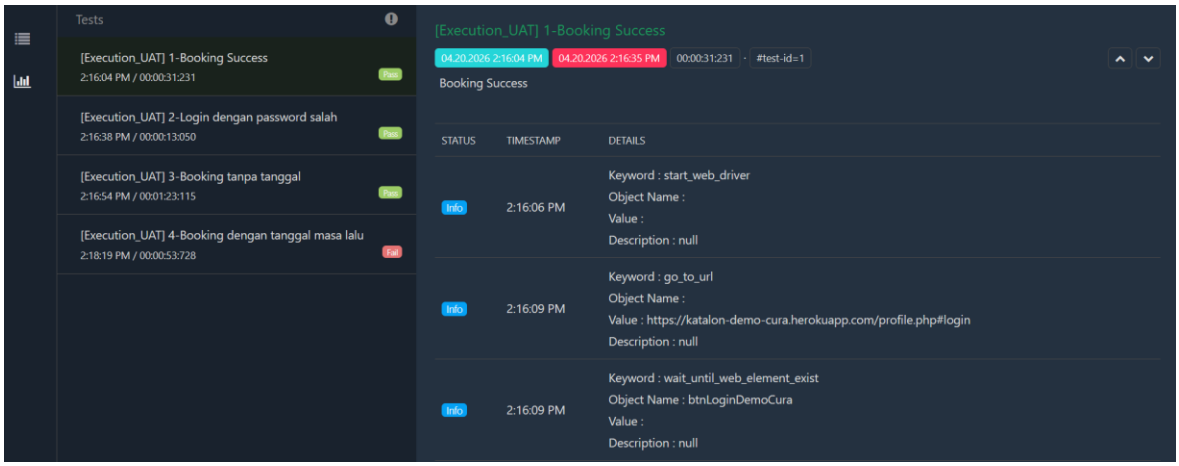
   The Extent Report provides a detailed and technical view of the automation execution, primarily used for debugging and analysis.

   It includes:

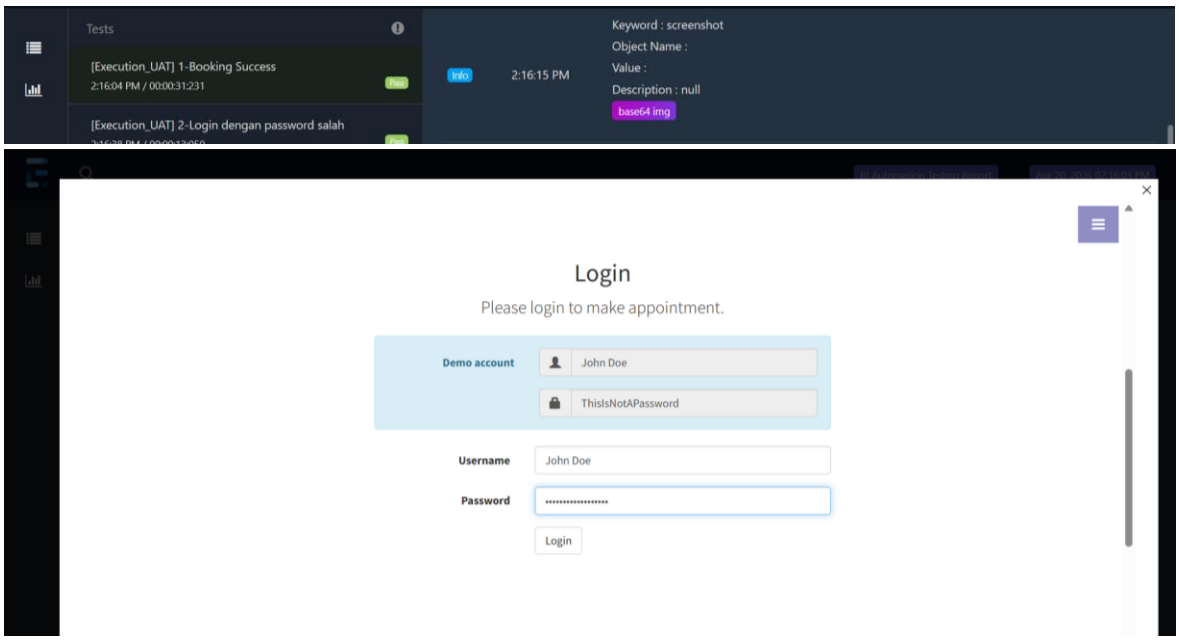
   - Test Summary
     Execution results per test case
   - Automation Logs
     Detailed step-by-step execution logs
   - Screenshots / Captures
     Visual evidence for each step, especially useful for failed scenarios




Test Summary




Automation Logs




Screenshots / Captures

**Conclusion**

The conclusion is structured into four key layers to provide a comprehensive overview of testing results, insights, and release readiness.

1. Summary Result
   Provides a high-level overview of test execution results, including:
   - Total test cases executed
   - Number of passed and failed tests
   - Overall pass rate
2. Insights
   Summarizes key findings from the testing process, such as:
   - The majority of test cases passed for core functionalities
   - Issues identified in date validation
   - The booking feature still has gaps in handling edge cases
3. Recommendations
   Provides actionable suggestions for improving application quality:
   - Implement stricter validation for date input
   - Perform additional testing on edge cases
   - Conduct regression testing after fixes are applied
4. Release Readiless
   Defines the release decision based on testing outcomes:
   - Ready, The application is ready for release with 100% test pass rate
   - Ready with Conditions, The application is ready for release with minor issues that do not impact core functionality or business operations
   - Not Ready, The application is not ready for release due to critical issues in core features

Testing Conclusion

Based on the automation testing results, out of a total of 4 test cases executed, 3 passed and 1 failed, resulting in a 75% pass rate.

The results indicate that most core functionalities, such as login and booking with valid data, are functioning as expected. However, a critical issue was identified in date validation, where the system still allows booking with a past date.

Key Findings

- Core functionalities are generally stable and working as expected
- A validation gap exists in the booking feature, specifically for past date input

Recommendations

- Implement stricter validation for the visit date field
- Perform regression testing after the issue is fixed

Release Decision: Not Ready

The application is currently not ready for release due to a defect that may impact business operations. Although the issue does not directly break the main application flow, it introduces a risk of incorrect user bookings, which could lead to poor user experience and potential service disruption.

Attachments, for supporting files, you can access the following link
https://drive.google.com/drive/folders/1_soPGVYqmxiRtxneUjocs_h5J9dcNdVn?usp=sharing